

LegalSathi AI

Technical Architecture Documentation

Version 1.0 · June 2026 · Confidential

- 1 Runtime & Framework
- 2 AI / LLM Stack
- 3 Authentication — Clerk v7
- 4 Database — PostgreSQL + pgvector
- 5 RAG Pipeline
- 6 Legal Domain System
- 7 Chat API — Streaming
- 8 Document Ingestion
- 9 Voice Architecture
- 10 Export System
- 11 Lawyer Directory
- 12 Frontend Components
- 13 Environment Variables
- 14 Build & Deployment
- 15 Full Data-Flow Diagram

1 Runtime & Framework

Layer	Technology	Version
Framework	Next.js	16.2.7
UI Runtime	React + React DOM	19.2.4
Language	TypeScript	5
Package Manager	pnpm	10.16.1
Node target	ES2017 / esnext bundler	—

Next.js 16 — Breaking Changes

- proxy.ts replaces middleware.ts — middleware is now exported as `proxy` with type ProxyConfig (not NextConfig).
- Server Actions body limit raised to 20 MB to support large legal document uploads.
- App Router with file-based routing, route groups (auth), catch-all segments `[...slug]`, and layouts.
- AGENTS.md and CLAUDE.md instruct agents to read `node_modules/next/dist/docs/` before writing any Next.js code.

TypeScript Configuration

- Strict mode enabled; path alias `@/* !` root directory.
- Includes `.next/types` and `.next/dev/types` for incremental type builds.
- Target: ES2017 — ensures broad Node.js compatibility.
- Special include: `docs/VoiceRecog.txt` (voice recognition reference documentation).

2 AI / LLM Stack

Google Gemini (@google/genai ^2.8.0)

Three distinct Gemini models are deployed for different cost/capability tradeoffs:

Model	Use Case	Rationale
gemini-2.5-flash	Main chat responses	Best quality; 8,192 output tokens
gemini-2.0-flash-lite	Title + suggestions (post-stream)	Higher free-tier RPM; avoids 429s
gemini-embedding-2	Document + query embeddings	768-dim vectors for pgvector
gemini-3.1-flash-live-preview	Real-time voice	Gemini Live WebSocket API

The two-model split (2.5-flash for chat, 2.0-flash-lite for title/suggestions) was introduced specifically to solve Gemini 429 rate-limit errors caused by 3 concurrent calls to the same model per message on the free tier. Both model names are overridable via GEMINI_MODEL and GEMINI_SECONDARY_MODEL environment variables.

Other LLM SDKs

- @anthropic-ai/sdk ^0.101.0 — Anthropic Claude, installed as optional fallback. Not wired into active flows.
- openai ^6.42.0 — Legacy OpenAI support. Present for compatibility; not used in active code paths.

Streaming Protocol

Chat responses are streamed via Server-Sent Events (SSE) using ReadableStream with Content-Type: text/event-stream. Each event is a JSON payload on a "data:" line followed by a blank line. Voice uses a persistent WebSocket to Gemini Live.

3 Authentication — Clerk v7.4.3

Architecture Overview

Clerk is the sole authentication provider. There is no username/password flow — only OAuth SSO via Google and Facebook.

```
Browser !' OAuth Provider (Google / Facebook)
!'/sso-callback (custom handler)
!'/handleRedirectCallback() + window.location.href (hard redirect)
!'/Next.js proxy.ts (clerkMiddleware validates JWT cookie)
!'/chat
```

proxy.ts — Middleware (Next.js 16 ProxyConfig)

Three route classes are defined using Clerk's createRouteMatcher:

Route Class	Pattern	Logic
isAuthRoute	/sign-in, /sign-up	Redirect to /chat if userId present
isProtectedRoute	/chat, /api/chat, /api/documents, ...	Redirect to /sign-in if no userId
isAdminRoute	/admin(.*)	Redirect to /chat if role `` admin

Admin role is read from sessionClaims.publicMetadata.role — set exclusively in the Clerk dashboard, never in client code.

SSO Callback — Hard Redirect

The OAuth callback page uses window.location.href instead of Next.js router.push() — this is critical. A soft Next.js navigation does not send a full HTTP request, so middleware never sees the fresh Clerk JWT cookie. The hard redirect forces a page reload, triggering middleware which then validates the session.

```
handleRedirectCallback(
  { signInForceRedirectUrl: '/chat', signUpForceRedirectUrl: '/chat' },
  (to) => { window.location.href = to; return Promise.resolve() }
)
```

JWT Lifecycle & Clock Skew

- Clerk session JWTs have a ~60-second TTL.
- `__client_uat` cookie signals an active session; `auth()` validates the short-lived JWT.
- Clock skew between development machine and Clerk servers causes nbf (not-before) failures.
- Fix on Windows: `w32tm /resync /force` (run as Administrator) to sync system clock via NTP.

`signIn.sso()` / `signUp.sso()` — Error Handling

In Clerk v7, these methods return `{ error }` — they do NOT throw. The catch block only handles network-level failures. Always destructure and check the returned error object.

```
const { error } = await signIn!.sso({ strategy, redirectCallbackUrl: '/sso-callback', redirectUrl: '/chat' })
if (error) {
  setError(error.longMessage ?? error.message ?? 'Something went wrong.')
  setLoading(null)
}
```

4 Database — PostgreSQL + pgvector

Connection Stack

- Neon Serverless PostgreSQL — DATABASE_URL env var.
- @prisma/adapters-pg wraps native pg driver for connection pooling on serverless.
- Prisma client uses global singleton pattern to prevent hot-reload re-initialization in dev.
- pgvector extension required: CREATE EXTENSION IF NOT EXISTS vector;

Schema Overview

User

- id (CUID primary key)
- clerkUserId (unique — mirrors Clerk identity)
- role enum: "user" | "admin"
- Relations: conversations, messages, documents, ingestionLogs

Conversation

- id (CUID)
- title (default: "New Chat")
- domain — one of: general | traffic | taxation | divorce | labor | property | business
- clerkUserId — ownership
- messages (1:many, cascade delete)

Message

- role: "user" | "assistant"
- content — full text of response or query
- conversationId — cascade delete

Document (Knowledge Base)

- status cycles: processing | ready | failed
- fileType: text | pdf | json | csv | manual
- domain — determines retrieval namespace
- chunks (1:many DocumentChunk)

DocumentChunk (Unit of Retrieval)

- content — ~400-word text segment
- embedding — vector(768), pgvector column; populated asynchronously after upload
- chunkIndex / totalChunks — ordering metadata

Lawyer (Advocate Directory)

- Auto-increment integer PK (not CUID)
- Fields: name, specialization, experience, location, languages[], feeRange, rating, casesHandled, bio, phone, email, featured
- Queried inline during chat when AI determines referral is needed

Vector Search Query Pattern

```
SELECT id, content, "documentId",  
       1 - (embedding <=> $1::vector) AS similarity  
FROM "DocumentChunk"  
WHERE "documentId" IN (
```

```
SELECT id FROM "Document" WHERE domain = $2 AND status = 'ready'
```



```
AND 1 - (embedding <=> $1::vector) > 0.45
```

```
ORDER BY embedding <=> $1::vector
```

LIMIT 10

The `<=>` operator is pgvector's cosine distance. `1 - distance` converts to similarity. Threshold 0.45 filters noise before RRF fusion.

5 RAG Pipeline

The RAG (Retrieval-Augmented Generation) pipeline is the core of LegalSathi's legal accuracy. It runs on every chat message before the prompt is sent to Gemini. Located in lib/rag.ts.

Step 1 — Domain Detection

Before retrieval, the query is scanned with regex signal patterns to detect cross-domain queries. Example: a traffic query mentioning "fine" or "tax" also triggers the taxation domain retrieval. Max 3 domains per query.

Step 2 — Query Expansion

A domain-specific keyword prefix is prepended to the user query before embedding to bias semantic search toward legal vocabulary:

Domain	Expansion Prefix
traffic	"Nepal motor vehicle traffic law rule fine violation "%5"é0" /"é\$"é/"é\$ "%o?"'ù."
taxation	"Nepal tax law IRD VAT PAN income tax 'Y0 ' ('
labor	"Nepal labor employment law worker minimum wage "iM" . ' ('
property	"Nepal land property registration stamp duty lalpurja 'É "Ù "â
business	"Nepal company OCR registration Industry Act "YM'ù>'©>"
divorce	"Nepal divorce family custody alimony Muluki Civil Code '©>" ?"Y>" ?'R

Step 3 — Dual Retrieval (per domain)

- Vector search — cosine similarity via embedding <=> operator on 768-dim Gemini embeddings.
- Full-text search — to_tsvector('simple', content) + plainto_tsquery, ranked by ts_rank_cd().
- Both run in parallel for each detected domain.

Step 4 — Reciprocal Rank Fusion (RRF)

Both ranked lists are merged using RRF with k=60:

score(chunk) = 1/(60 + rank_vector) + 1/(60 + rank_fts)

RRF is a parameter-free fusion method that is robust to score scale differences between semantic and lexical retrieval. k=60 is the standard default from the original RRF paper (Cormack et al., 2009). Results from multiple domains are interleaved (primary domain first at each round).

Step 5 — Budget & Filtering

Parameter	Value	Purpose
Similarity threshold	0.45	Filter low-quality vector matches before RRF
Max chunks returned	6	Keeps prompt token count manageable
Max context words	2,500	Hard word budget across all retrieved chunks
RRF k parameter	60	Standard RRF constant, controls rank influence

This section explains every technical term used in the RAG pipeline from first principles, so any reader can understand how retrieval-augmented generation works in LegalSathi.

Retrieval-Augmented Generation (RAG) is an architecture that combines a search engine with a language model. Instead of relying solely on what the LLM learned during training, RAG fetches real documents at query time and injects them into the prompt. The model then answers using that retrieved evidence rather than memory alone. In LegalSathi, the "documents" are chunks of Nepalese legal statutes, rules, and guidelines stored in PostgreSQL.

Embeddings & Vector Space

An embedding is a list of numbers (a vector) that represents the meaning of a piece of text. Similar meanings produce vectors that are numerically close to each other in high-dimensional space. LegalSathi uses Google's gemini-embedding-2 model to produce 768-dimensional embeddings — each chunk of legal text becomes a point in 768-dimensional space.

- Embedding dimension: 768 — each document chunk is represented by exactly 768 float numbers.
- Training: Gemini embedding model is trained on massive corpora so "fine" and "penalty" are near each other.
- Stored in: PostgreSQL column of type vector(768) via the pgvector extension.
- Generated once per chunk at ingest time; generated fresh per query at query time.

Why 768 dimensions? Gemini embedding-2 uses 768 as its output size. More dimensions = finer meaning resolution, but more storage and slower distance computation. 768 is the standard BERT-class size — a proven tradeoff.

Cosine Similarity

Cosine similarity measures how similar two vectors are by computing the cosine of the angle between them. A value of 1.0 means the vectors point in exactly the same direction (identical meaning). A value of 0.0 means they are perpendicular (completely unrelated). Negative values mean opposite meanings (rare in embedding models).

Range: -1.0 (opposite) !' 0.0 (unrelated) !' 1.0 (identical)

pgvector exposes this as the `<=>` operator which computes cosine `DISTANCE = 1 - cosine_similarity`. So lower `<=>` values = more similar. LegalSathi converts back: `similarity = 1 - distance`, and filters out anything below 0.45 (less than 45% similar to the query).

Extremely close semantic match

Top-ranked; always included

Strong topical match

High priority in RRF merge

0.45 – 0.7

Moderate relevance

Included if within chunk budget

< 0.45

Weak or unrelated

Filtered out before RRF

BM25 — Best Match 25

BM25 (Best Match 25) is a classical probabilistic ranking algorithm for full-text search. It scores documents based on how many times query terms appear, weighted by how rare those terms are across the corpus (IDF — Inverse Document Frequency) and normalized for document length.

BM25 score for term t in document d :

$$\text{score}(d, t) = \text{IDF}(t) \times \left[\text{TF}(t,d) \times (k_1 + 1) \right. \\ \left. \left[\text{TF}(t,d) + k_1 \times (1 - b + b \times |d|/\text{avgDL}) \right] \right]$$

Where:

- $\text{TF}(t,d)$ = term frequency (how many times t appears in d)
- $\text{IDF}(t)$ = $\log((N - \text{df} + 0.5) / (\text{df} + 0.5) + 1)$
 N = total docs, df = docs containing t
- k_1 = saturation param (typically 1.2 – 2.0)
- b = length normalization param (typically 0.75)
- $|d|/\text{avgDL}$ = ratio of document length to average doc length

In PostgreSQL, BM25-like scoring is approximated using `ts_rank_cd()` on `tsvector` columns. LegalSathi creates a `tsvector` from each chunk's content using the "simple" dictionary (no stemming, preserves Nepali/legal terms exactly), then ranks matches with `ts_rank_cd()`.

Why not pure BM25? BM25 is great for keyword overlap (exact statute section numbers, legal terms) but fails for paraphrase ("can I be fired?" vs "wrongful termination"). Cosine similarity handles paraphrase but can miss exact keyword matches. Combining both captures the strengths of each.

Vector Search vs Full-Text Search — Comparison

Aspect	Vector (Cosine)	Full-Text (BM25-style)
Matches	Semantic meaning / paraphrase	Exact keywords and legal terms
Strength	"How do I divorce?" finds "divorce procedure"	"Section 94" finds "Section 94"
Weakness	Can miss exact statute numbers	Cannot handle paraphrase or synonyms
Query type	Dense (768-float query vector)	Sparse (keyword tokens)
Speed	ANN index scan (pgvector)	Inverted index (GIN index on tsvector)
Score scale	0.0 – 1.0 (cosine similarity)	Unbounded float (ts_rank_cd)

Reciprocal Rank Fusion (RRF)

RRF is a rank-based fusion algorithm that merges two (or more) ranked lists into a single combined ranking. It was introduced by Cormack, Clarke, and Buettcher in 2009 and has become the standard for hybrid search. The key insight: instead of trying to normalize incompatible score scales (cosine vs BM25), RRF only uses the RANK position of each document in each list.

```
For each document d that appears in any ranked list:

    RRF_score(d) = sum over each list L of:  1 / (k + rank_L(d))

In LegalSathi (k = 60, two lists):

    RRF_score(chunk) =  1          +          1
                        % % % % % % % % % %      % % % % % % % % % %
                        60 + rank_vec    60 + rank_fts

    rank_vec = position in cosine similarity ranking (1 = most similar)
    rank_fts = position in BM25/ts_rank_cd ranking (1 = highest keyword score)
    k = 60      prevents top-ranked items from dominating too heavily
```

A chunk that appears at rank 1 in cosine search contributes $1/(60+1) = 0.0164$. At rank 10 it contributes $1/(60+10) = 0.0143$. The difference shrinks quickly — this is why $k=60$ is "soft": a rank-1 result is important but doesn't completely drown out rank-2. Chunks absent from one list get no contribution from that list (treated as if $\text{rank}=""$).

Why k = 60?

The original RRF paper empirically found $k=60$ to be robust across many retrieval tasks. Smaller k (e.g. $k=1$) makes the formula very aggressive — rank 1 scores much higher than rank 2. Larger k (e.g. $k=1000$) flattens all ranks toward equal weight, losing ordering signal. $k=60$ is the standard choice adopted across most production hybrid search systems.

Hybrid Search — Why Both?

Consider this real LegalSathi query: "mapase fine katiko cha" (how much is the DUI fine?)

Search Type	What It Finds	Score
Cosine (vector)	Chunks about drunk driving penalties <i>(semantic match)</i>	High
BM25 (FTS)	Chunks containing the exact phrase "mapase" or "fine"	High
Cosine only (no BM25)	Misses chunks that just list fine amounts <i>(low semantic density)</i>	Lower
BM25 only (no vector)	Misses chunks that say "DUI" or "alcohol"	

violation" instead

Lower

RRF combined

Gets both — statute text + penalty table

Best

Embeddings in LegalSathi — End-to-End

Here is the complete lifecycle of an embedding in the system:

```
INGEST TIME:
Document text
  !' chunkText()          !' ["chunk 1 text", "chunk 2 text", ...]
  !' generateEmbeddingsBatch(chunks)
    !' Gemini embedding-2 API (50 chunks per call)
    !' [[0.12, -0.04, 0.87, ... 768 floats], ...]
  !' INSERT INTO DocumentChunk (content, embedding)
    !' stored as PostgreSQL vector(768)

QUERY TIME:
User query: "DUI fine Nepal"
  !' Query expansion: "Nepal motor vehicle traffic law DUI fine "%5"é0" âââ
  !' generateEmbedding(expandedQuery)
    !' Gemini embedding-2 API (single call)
    !' [0.09, -0.01, 0.91, ... 768 floats]
  !' pgvector: ORDER BY embedding <=> queryVector
    !' returns top chunks sorted by cosine distance
```

Cross-Domain Retrieval

A single query may span multiple legal domains. LegalSathi detects this with regex signal patterns and runs retrieval in parallel across up to 3 domains, then interleaves results (primary domain first).

```
Query: "bike chalau fine tireko thiyo, VAT rakha ra?"
      (I paid a traffic fine, do I keep the VAT receipt?)

Domain signals detected:
  !' traffic (fine, bike)
  !' taxation (VAT, receipt)

Parallel retrieval:
  traffic domain !' vector search + BM25 !' top 6 chunks !' RRF list A
  taxation domain !' vector search + BM25 !' top 6 chunks !' RRF list B

Interleave:
  [traffic#1, taxation#1, traffic#2, taxation#2, ...] !' first 6 fit budget
```

Chunking & Overlap — Why It Matters

Legal text often has context that spans paragraph boundaries. A section header and its first clause may be in different chunks if split naively. LegalSathi uses a 2-sentence overlap to mitigate this:

```
Chunk 1: "Section 94. Divorce by mutual consent. Either spouse may..."
        "...file a petition jointly at the District Court."      !• overlap !'

Chunk 2: "...file a petition jointly at the District Court."      !• carried over
        "Section 95. Contested divorce. Grounds include..."
        "...mental illness, cruelty, abandonment for 3 years."
```

Without overlap, a query about filing a divorce petition might only match Chunk 2 (which contains "file a petition") but miss that it requires joint filing. The overlap ensures that critical cross-boundary context is duplicated into adjacent chunks.

tsvector & plainto_tsquery — PostgreSQL FTS

PostgreSQL's full-text search stores pre-processed text as a tsvector (a sorted, de-duplicated list of lexemes). At query time, plainto_tsquery converts the user's query into a tsquery (ANDed terms by default). ts_rank_cd() then scores each matching tsvector by term frequency and proximity.

```
-- What LegalSathi stores per chunk:
to_tsvector('simple', content)
-- 'simple' dictionary: no stemming, lowercase only
-- Preserves: "Section", "94", "NPR", "VAT", "lalpurja" exactly

-- At query time:
plainto_tsquery('simple', 'divorce mutual consent')
-- produces: 'divorce' & 'mutual' & 'consent'

-- Ranking:
ts_rank_cd(tsvector_col, query)
-- cd = cover density: rewards chunks where query terms appear close together
```

Why "simple" dictionary and not "english"? The "english" dictionary applies stemming (removes '!' remov, filing !' file) which would corrupt Nepali words, section numbers, and legal terms like "lalpurja", "malpot", "IRD". The "simple" dictionary only lowercases, preserving all tokens exactly.

Summary — RAG Term Glossary

Term	One-Line Definition
RAG	Fetch relevant docs first; give them to the LLM as context
Embedding	List of numbers (vector) representing text meaning
Vector space	768-dimensional coordinate system where meaning = proximity
Cosine similarity	Angle between two vectors; 1.0 = same meaning, 0.0 = unrelated

PostgreSQL extension that stores and queries vectors natively

`<=>` operator

pgvector cosine distance: 0.0 = identical, 2.0 = opposite

Keyword ranking by term freq $\times$ rarity; foundation of full-text search

tsvector

plainto_tsquery

Converts a plain string into an ANDed PostgreSQL text query

ts_rank_cd

Merge two ranked lists by position, not score; $k=60$ softens dominance

Hybrid search

Run vector + BM25 in parallel, fuse with RRF

Chunking

Split long docs into ~400-word segments for embedding

Overlap

2-sentence carry-over between chunks to preserve cross-boundary context

Query expansion

Prepend domain keywords to query before embedding for better recall

Similarity threshold

0.45 minimum cosine score to include a chunk at all

Word budget

2,500 max words of retrieved context injected into the prompt

Step 6 — System Prompt Assembly

`buildSystemPrompt(chunks, domain)` wraps retrieved chunks in `<retrieved_legal_context>` XML tags, prepends the domain's `systemInstructions`, and appends the RESPONSE QUALITY MANDATE:

- CITE SPECIFIC SECTIONS — exact act and section (e.g. "MVTMA 2049, Section 164(1)")
- GIVE EXACT AMOUNTS — specific NPR figures or tight ranges
- GIVE EXACT TIMELINES — real durations from the law
- COVER ALL ANGLES — procedure, cost, documents, legal consequence, contact
- DISTINGUISH SOURCES — prefix pre-trained knowledge with "Based on Nepal law:"
- NO HEDGING ON FACTS
- MINIMUM DEPTH — at least 3 substantive items per section
- PLAIN LANGUAGE — translate legal jargon immediately

6 Legal Domain System

7 domains are defined in lib/domains.ts. Each has: slug, label, icon (emoji), description, and systemInstructions.

Statute References Per Domain

Domain	Primary Statutes
Traffic	Motor Vehicles & Transport Management Act 2049 (MVTMA)
Taxation	Income Tax Act 2058, Value Added Tax Act 2052, IRD Guidelines
Divorce/Family	Muluki Civil Code 2074 Part 4 (§93–176), DV Act 2066, Children's Act 2075
Labor	Labor Act 2074, Labor Rules 2075, Social Security Act 2075, Bonus Act 2030
Property	Land Registration Act 2021, Lands Act 2021, Land Revenue Act 2034
Business	Company Act 2063, Industrial Enterprises Act 2076, FITTA 2075, VAT Act 2052
General	Constitution of Nepal 2072, Muluki Criminal Code 2074, all of the above

Key Statutory Numbers Embedded

Labor Act 2074

- Written contract required within 15 days (§8)
- Probation: max 6 months, cannot extend (§13)
- Employer termination: 30 days notice (§28); Employee: 30 days notice (§29)
- Wrongful termination: 1 month/year of service, min 3 months compensation (§33)
- Home leave: 13 days/year (§52); Sick leave: 12 days/year (§53)
- Maternity: 98 days (§55); Paternity: 15 days (§56)
- Overtime: max 4h/day, 24h/week, 1.5× rate (§45)
- Gratuity: 8.33% of basic salary per year of service (§59)
- Minimum Wage (2082 BS): NPR 19,550/month

Property Law

- Stamp duty: 4% of government valuation (male buyer); 1.5% (female buyer)
- Capital gains: 5% if held >5 years; 10% if <5 years; 15% corporate
- Tenancy risk: 3+ years occupation may trigger legal claim (Lands Act 2021 §26)
- Land ceiling — Kathmandu Valley: max 10 ropani

Business Registration

- OCR registration fee: NPR 1,000–5,000 (depends on paid-up capital)
- VAT registration threshold: NPR 50 lakh annual turnover (VAT Act §10)
- Corporate tax: 25% standard; 20% manufacturing; 15% SEZ; 1% cooperatives
- Trademark: Department of Industry, IP Division — 10-year validity, renewable

Domain Gating & Anti-Hallucination

- Each domain's systemInstructions includes an explicit out-of-scope rejection template.
- Traffic domain refuses divorce/labor queries with a hard-coded redirect message.
- All domains forbid inventing amounts, section numbers, deadlines, or procedures.
- If context is insufficient, the domain must state it — not guess.

Local Vocabulary Mapping (Traffic Domain)

Colloquial	Legal Meaning
Mama / Traffic Mama / Cop	Nepal Traffic Police officer
Chit / Chalan / Cheet / Slip	Official traffic fine / ticket
Mapase	Drunk driving (DUI)
Bluebook / Bilbuk	Vehicle Registration Certificate
Yatayat	Department of Transport Management (DOTM)
Dalal	Informal broker / middleman
Likhit	Written driving license examination
Trial	Practical driving test

7 Chat API — Streaming Architecture

POST /api/chat — Full Flow

```
Request: { conversationId, message }
1. requireAuth() - validate Clerk JWT from cookie
2. Save Message (role=user) to DB
3. RAG retrieval - retrieveRelevantChunks(message, domain)
4. buildSystemPrompt(chunks, domain)
5. generateContentStream(prompt) !' Gemini 2.5-flash (8,192 token max output)
6. ReadableStream !' SSE to browser
[Post-stream, non-blocking Promise.resolve()]
7. Save Message (role=assistant) to DB
8. Generate title !' gemini-2.0-flash-lite (first message of conversation only)
9. Generate suggestions !' gemini-2.0-flash-lite
```

SSE Event Types

Event	Payload	When Sent
text	{ text: string }	Each streamed token from Gemini
sources	{ sources: Source[] }	Once; after first token; document refs
scenario	{ scenario: ScenarioJSON }	When AI returns structured JSON mode
suggestions	{ suggestions: string[] }	Post-stream; secondary model call
error	{ error: string }	Any failure; user-facing friendly message
[DONE]	literal string	Stream complete; client closes connection

Structured JSON (Scenario) Mode

When the AI determines a structured response is appropriate it returns a JSON blob inside the text stream. The API parses it and re-emits as a "scenario" event. The client renders it via <ScenarioRenderer>.

Response Modes

- violation — traffic or regulatory infraction with fine, points, procedure
- procedure — step-by-step legal process (registration, filing, appeal)
- information — factual legal information (statute explanation,

- rights)
- rights — constitutional or statutory rights of the user
 - emergency — urgent situation requiring immediate legal action

UI Variants (per section)

Variant	Rendering
stepper / timeline	Numbered steps with cost/duration badges
card_grid	2-column grid with tag chips
table	Alternating-row styled table
stat_grid	Metric + label + note layout
checklist	Checkbox items with descriptions
alert_box	Yellow (high severity) / blue (low) left-border boxes
info_banner	Headline + description highlight box
comparison	3-column aspect comparison
minimal_text	Plain text — fallback

Lawyer Matching (inline)

After the AI generates a scenario with `required_lawyers[]`, the chat API immediately queries the Lawyer table by specialization and attaches matching real advocates directly to the SSE scenario event. This gives users a concrete referral alongside the legal guidance in a single response.

8 Document Ingestion Pipeline

POST /api/documents — Async Flow

Body: { title, content, fileType?, domain? }

1. Create Document row – status: "processing"
2. Return 202 Accepted immediately (non-blocking)
3. Background (unawaited Promise):
 - a. chunkText(content)
! ' sentence-aware splitting, ~400 words/chunk, 2-sentence overlap
! ' discard chunks < 30 words
 - b. Bulk INSERT DocumentChunk rows (content only; embedding = null)
! ' batches of 50 rows (connection pool safety)
 - c. generateEmbeddingsBatch(allChunkTexts)
! ' 50 texts per Gemini call
! ' Exponential backoff retry: 6 attempts; handles 429 + 5xx
 - d. UPDATE DocumentChunk SET embedding = \$1 WHERE id = \$2
! ' batches of 50
 - e. UPDATE Document SET status = 'ready'

Chunking Algorithm (lib/embeddings.ts)

- Splits on sentence boundaries: (?<=[!?!])\s+(?=[A-Z]) and blank lines (?<=\n)\n+
- Accumulates sentences until reaching target ~400 words
- Carries last 2 sentences as overlap into next chunk (context continuity)
- Discards chunks < 30 words (avoids embedding noise)

Embedding Batch & Retry

- Batch size: 50 texts per Gemini embedding-2 call
- 6 max attempts with exponential backoff
- Retries on: 429 Too Many Requests, 5xx server errors
- Replaces newlines with spaces before embedding (API requirement)

Re-Embed Endpoint POST /api/documents/[id]/re-embed

For documents with partially failed embedding (some chunks have embedding=null): queries null-embedding chunks only, re-runs batch embedding, updates vectors, marks document ready. Returns 202 and runs async. Useful for recovering from Gemini quota exhaustion during initial ingest.

Client-Side File Processing

- Supported: .txt, .md, .json, .csv, .pdf

- PDF text extraction: pdfjs-dist ^6.0.227 runs entirely in the browser
- Automatic chunk count estimation shown before upload
- Drag-and-drop + bulk upload queue in app/ingest/page.tsx

Polling for Processing State

The ingest page polls GET /api/documents/[id] every 3 seconds for documents in "processing" state. Fix for infinite 404 loop: 404 responses set a "missing" flag; those document IDs are filtered out of React state rather than staying in the polling queue indefinitely.

```
const missingIds = new Set(results.filter(r => r.missing).map(r => r.id))
setDocuments(prev =>
  prev
    .filter(d => !missingIds.has(d.id)) // remove 404'd docs
    .map(d => { /* update status from API response */ })
)
```

9 Voice Architecture

Four-layer stack. All layers are client-side except the RAG context fetch which hits the server.

Layer 1 — Audio Capture (lib/voice/audioCapture.ts)

- Wraps Web SpeechRecognition API (Chrome/Edge native)
- Language: ne-NP (Nepali with English code-switching)
- Continuous mode with interim results for real-time display
- Auto-restarts on timeout — Chrome disconnects after ~60s silence
- Callbacks: onTranscript, onSpeechStart, onSpeechEnd

Layer 2 — RAG Context Injection (POST /api/voice-rag)

Before each user turn is sent to Gemini Live, the current transcript is sent to /api/voice-rag. The server runs `retrieveRelevantChunks(query, domain)` — the same pipeline as text chat — and returns `{ context: string }` formatted source text. This is prepended to the Gemini Live turn, giving the voice assistant the same legal accuracy as text chat.

Layer 3 — Gemini Live (lib/voice/geminiLiveClient.ts)

- Model: gemini-3.1-flash-live-preview
- Uses `@google/genai v1beta live.connect()` for WebSocket connection
- Modalities: audio output + transcription enabled
- Voice: "Aoede" with Nepali language hint
- Protocol: `endUserTurn(transcript) !' inject RAG context !' send with turnComplete=true`

Layer 4 — Audio Playback (lib/voice/audioPlayback.ts)

- Web Audio API — `AudioContext` at 24 kHz (matches Gemini Live PCM16 output)
- Base64 !' PCM16 !' Float32 !' `AudioBuffer` decoding pipeline
- Queue-based: auto-chains audio chunks as they arrive from WebSocket
- Barge-in support: immediately clears queue and stops current playback

Full Voice Flow

```
User speaks
!' SpeechRecognition (ne-NP) [continuous, interim results]
!' onTranscript callback
!' POST /api/voice-rag !' RAG retrieval !' { context }
!' GeminiLiveClient.endUserTurn(transcript)
!' inject context into Gemini Live turn
!' send turn with turnComplete=true (WebSocket)
!' Gemini Live server response:
  %% outputTranscription (AI text for display)
  %% audio chunks        (base64 PCM16)
  %% turnComplete         (end of response)
!' AudioPlayback.queueChunk(pcm16)
!' Web Audio API playback @ 24 kHz
```

10 Export System

Format	Generated Where	Mechanism
Markdown	Client-side	String template from scenario JSON
PDF	Client-side	Markdown !' HTML !' window.print()
DOCX	Server-side (/api/export)	docx library (Office Open XML)

Markdown (lib/export.ts — scenarioToMarkdown)

- Renders each scenario section with variant-specific formatting
 - Stepper !' numbered list; table !' markdown table; checklist !' task list
 - Includes: summary, map entities, legal references, sources, disclaimer
 - Mode: "full" (all sections) or "brief" (high-priority only)

PDF (lib/export.ts — scenarioToHtml)

- Converts markdown output to HTML with print-optimized CSS
 - Typography: Georgia serif, narrow margins, A4 page breaks
 - Injects into a new browser window and auto-triggers window.print()
 - Page-break hints for tables and long sections

DOCX (app/api/export/route.ts + docx ^9.7.1)

- Uses Document API from docx (Office Open XML format)
 - Styled: headings, tables with alternating rows, alert boxes with colored left borders
 - Metadata: creator="LegalSathi", title=scenario.title
 - Returns binary buffer with Content-Disposition: attachment; filename=....docx

11 Lawyer Directory

Schema: Lawyer Table

Field	Type	Purpose
id	Int (auto-increment)	Primary key — integer not CUID
name	String	Full name of advocate
specialization	String	Field of law (civil, criminal, tax, etc.)
experience	Int	Years of practice
location	String	City / district
languages	String[]	Languages spoken
feeRange	String	e.g. "NPR 1,000–5,000/hr"
rating	Float	Star rating
featured	Boolean	Premium listing flag
phone / email	String	Contact details

Search API (GET /api/lawyers)

- Full-text search on name, specialization, bio, location — PostgreSQL ILIKE
- Filters: specialization, location, minRating, featured (boolean)
- Pagination: limit / offset query params
- Returns array of Lawyer objects ordered by featured desc, rating desc

Inline Chat Matching

After scenario generation, the chat API reads required_lawyers[] from the JSON response and immediately queries the Lawyer table by specialization. Matching advocates are attached to the SSE scenario event so users receive a concrete referral in the same response — no extra round-trip needed.

12 Frontend Components

ScenarioRenderer.tsx

The primary output renderer for structured AI responses (~500 lines). Accepts a parsed ScenarioJSON object and renders each section with a variant-specific component. Also renders the map entities section (government office images), required lawyers (from DB), and summary.

LegalResponse.tsx

Markdown-to-React converter for plain-text (non-scenario) responses. Custom renderers:

- Ordered lists !' navy-circled numbered step cards
- Unordered lists !' styled bullet dots
- Strong text !' navy colour emphasis
- H2 / H3 headings !' section dividers with border-bottom
- Tables !' styled markdown tables

VoiceAssistant.tsx

Orchestrates the full voice loop (microphone !' transcription !' RAG !' Gemini Live !' audio playback). Renders conversation transcript with user and AI turns, session status indicator, and start/stop/clear controls. Props: domain, autoStart, compact, onClose.

app/chat/page.tsx (~1,200 lines)

The main application UI. Key sections:

- Left sidebar: brand, navigation, new conversation button, conversation history, knowledge base panel
- Main area: message thread (user navy bubbles + AI avatar), streaming indicator, sources chips, suggestion buttons
 - Input: growing textarea (max 300px), voice toggle, send button
 - Hero mode: domain selection grid when no conversation is active
 - Export dropdown: md, docx, pdf × brief/full modes

Landing Page Components

- Navbar, HeroSection, ProblemSection, SolutionSection, FeaturesSection
- HowItWorksSection — RAG pipeline explanation for non-technical users
- DemoSection, ComparisonSection, FutureVisionSection, ImpactSection, CtaBanner, Footer

13 Environment Variables

```
# PostgreSQL (required)
DATABASE_URL=postgresql://user:pass@host/db?sslmode=require

# Google Gemini (required)
GEMINI_API_KEY=AIza...
GEMINI_MODEL=gemini-2.5-flash
GEMINI_SECONDARY_MODEL=gemini-2.0-flash-lite
NEXT_PUBLIC_GEMINI_API_KEY=AIza...    # client-side voice only

# Clerk Authentication (required)
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=pk_test...
CLERK_SECRET_KEY=sk_test...
NEXT_PUBLIC_CLERK_SIGN_IN_URL=/sign-in
NEXT_PUBLIC_CLERK_SIGN_UP_URL=/sign-up
NEXT_PUBLIC_CLERK_SIGN_IN_FORCE_REDIRECT_URL=/chat
NEXT_PUBLIC_CLERK_SIGN_UP_FORCE_REDIRECT_URL=/chat
CLERK_WEBHOOK_SECRET=whsec...
```

NEXT_PUBLIC_GEMINI_API_KEY is exposed to the browser for the Gemini Live voice WebSocket. Scope it to voice-only; never use it for server-side RAG or embedding calls.

14 Build & Deployment

Commands

```
# Install dependencies (triggers postinstall: prisma generate)
pnpm install

# Development
pnpm dev

# Production build
prisma generate && next build

# Start production server
next start
```

Key Build Notes

- postinstall runs prisma generate automatically — Prisma client types regenerated from schema.prisma on every install.
- next build compiles both server and client bundles; App Router generates RSC manifests.
- Target platform: Vercel (native Next.js) or any Node.js 18+ host.
- DATABASE_URL must point to a running PostgreSQL instance with pgvector extension at build time (Prisma introspection).

Critical Dependencies & Versions

Package	Version	Purpose
@google/genai	^2.8.0	Chat, embeddings, Gemini Live voice
@prisma/client	^7.8.0	ORM for PostgreSQL
@prisma/adapters-pg	^7.8.0	Native pg connection pooling adapter
@clerk/nextjs	^7.4.3	Auth — user roles, session management
docx	^9.7.1	Server-side DOCX generation
pdfjs-dist	^6.0.227	Client-side PDF text extraction
react-markdown	^10.1.0	Markdown rendering in chat
framer-motion	12.40.0	UI animations
tailwindcss	^4	Styling via PostCSS plugin
@anthropic-ai/sdk	^0.101.0	Optional Claude fallback

15 Full Data-Flow Diagram

Text Chat Flow

```
USER TYPES QUESTION
% % POST /api/chat
% % requireAuth() - Clerk JWT from cookie (middleware already validated)
% % Save Message (role=user) to PostgreSQL
% % RAG Pipeline:
%   % % Domain detection           (regex signal scan)
%   % % Query expansion            (prepend domain keywords + Nepali terms)
%   % % generate query embedding   (Gemini embedding-2, 768-dim)
%   % % Vector search              (pgvector cosine <=>; threshold 0.45)
%   % % Full-text search           (tsvector + plainto_tsquery; ts_rank_cd)
%   % % RRF fusion k=60            !' top 6 chunks, max 2,500 words
% % Build system prompt            (domain rules + retrieved context XML)
% % generateContentStream          !' Gemini 2.5-flash (8,192 token max)
% % SSE ReadableStream to browser:
% % { text }                      - token by token
% % { sources }                   - retrieved document chips
% % { scenario }                  - parsed JSON !' ScenarioRenderer
% % { suggestions }               - follow-up questions
% % [DONE]
[Post-stream, non-blocking]:
% % Save Message (role=assistant) to PostgreSQL
% % gemini-2.0-flash-lite !' generate conversation title
% % gemini-2.0-flash-lite !' generate 3 follow-up suggestions
```

Document Ingestion Flow

```
FILE UPLOADED (drag-drop or paste in /ingest)
% % Client-side: PDF? !' pdfjs-dist text extraction !' raw text string
% % POST /api/documents !' 202 Accepted (immediate)
[Background async task]:
% % chunkText()
%   % % Split on sentence boundaries + blank lines
%   % % Target: ~400 words/chunk; 2-sentence overlap
%   % % Discard chunks < 30 words
% % INSERT DocumentChunk rows (batch 50) - embedding=null
```

```
% % generateEmbeddingsBatch()
```

% % 50 texts per Gemini embedding-2 call

% % Retry: 6 attempts, exponential backoff (429 + 5xx)

```
% % UPDATE DocumentChunk SET embedding=$1 (batch 50)
```



```
% % UPDATE Document SET status='ready'
```


FRONTEND polls GET /api/documents/[id] every 3s

% % 404? !' filter document out of React state (prevents infinite 404 loop)

```
% % ready? !' stop polling; show chunk count
```

Voice Session Flow

USER SPEAKS

```
% % SpeechRecognition API (ne-NP, continuous)
% % onTranscript(text) callback fires
% % POST /api/voice-rag { query: text, domain }
% % retrieveRelevantChunks() !' same RAG as text chat
% % returns { context: string }
% % GeminiLiveClient.endUserTurn(transcript)
% % prepend RAG context to message content
% % send over WebSocket with turnComplete=true
% % Gemini Live WebSocket response:
% % outputTranscription !' display AI text in transcript
% % audio chunks (base64 PCM16)
% % AudioPlayback.queueChunk()
% % Decode PCM16 !' Float32 !' AudioBuffer
% % Web Audio API playback @ 24 kHz
% % turnComplete !' ready for next user turn
```

Authentication Flow

USER CLICKS "Continue with Google"

```
% % signIn.sso({ strategy: 'oauth_google', redirectCallbackUrl: '/sso-callback' })
% % Clerk redirects !' Google OAuth consent screen
% % Google redirects !' /sso-callback?code=...&state=...
```

/sso-callback (page.tsx):

```
% % handleRedirectCallback(options, navigate)
% % navigate = (to) => { window.location.href = to } !• HARD REDIRECT
% % Full HTTP request !' Next.js proxy.ts (clerkMiddleware)
% % auth() validates fresh JWT cookie
% % isAuthRoute check: userId present !' redirect /chat
% % /chat renders (user is now authenticated)
```


